

Executive Summary Welcome to the world of backend routing! In this video, we break down how a backend server figures out exactly what to do when a request comes in. If HTTP methods (like **GET** or **POST**) describe *what* action we want to take, routing describes *where* we want that action to happen. By the end of this guide, you'll understand how servers map incoming URLs to the specific code (handlers) that processes them, ensuring every request finds its way home.

What is Routing? The "Where" of a Request

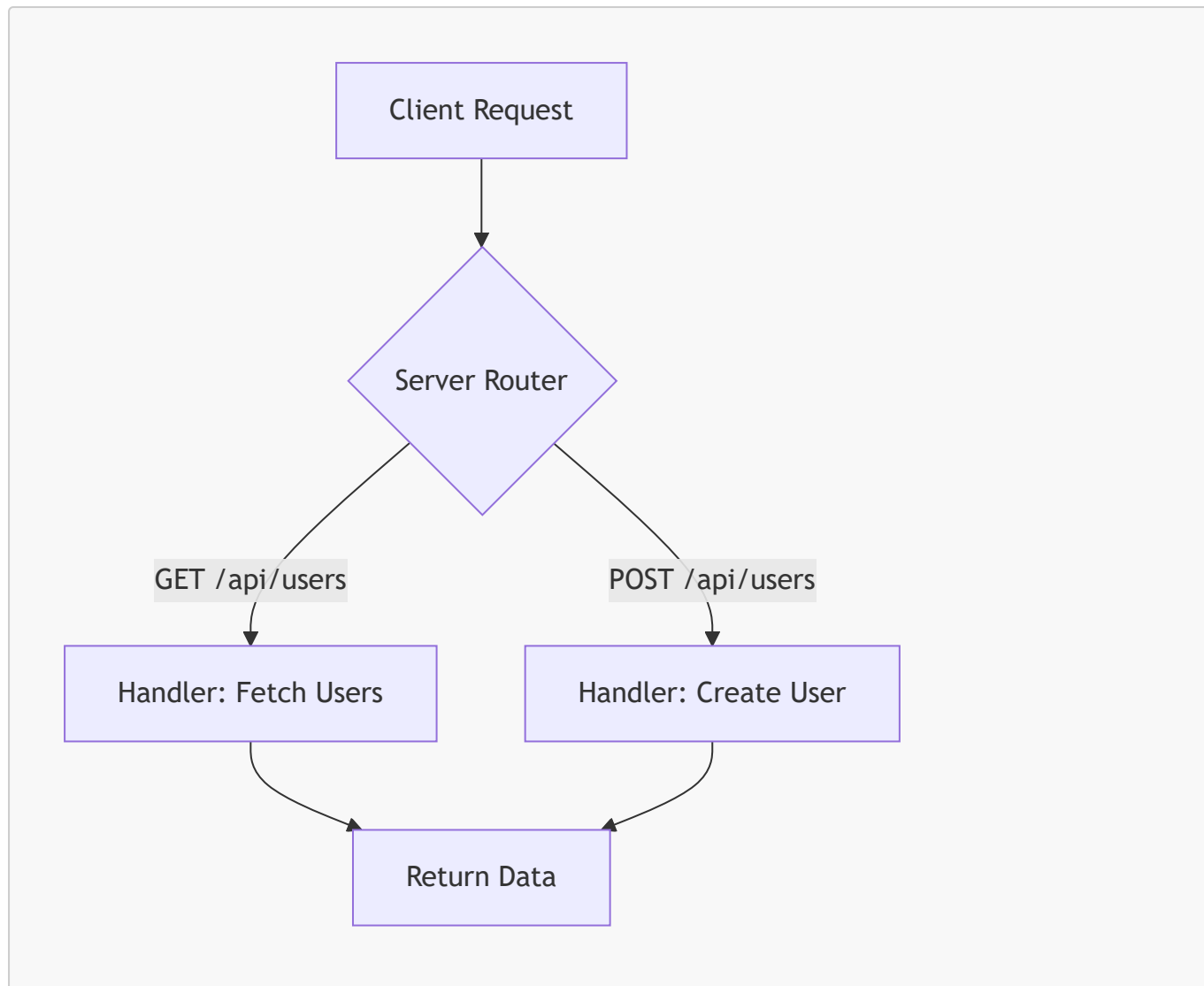
Hey there! If you remember from our earlier discussions on HTTP methods, methods like **GET**, **POST**, or **DELETE** express your **intent** or **action**. They tell the server *what* you want to do.

Routing, on the other hand, expresses the **where**. When you want to fetch or save data, you need to tell the server exactly *which* resource you are targeting. Think of it like a mailing address. The HTTP method is the instruction (e.g., "deliver this package"), and the route is the physical address (e.g., "123 Main St").

When your server receives a request, it looks at two things:

1. The **HTTP Method** (The Action)
2. The **URL Path** (The Address/Route)

The server combines these two pieces of information to create a unique key. It then maps this key to a specific block of code—called a **Handler**—which runs your business logic, talks to the database, and sends back a response.



Static Routes

Let's start with the simplest form of routing: **Static Routes**.

A static route is a constant URL path that never changes. It doesn't have any variable or moving parts inside of it. For example, a route like `/api/books` is static because that exact string is always used to target the books resource.

When you send a `GET` request to `/api/books`, the server maps it to the handler that returns a list of books. If you send a `POST` request to that exact same static route, the server maps it to a completely different handler designed to create a new book. The path is identical, but the HTTP method safely differentiates the two.

Feature	GET /api/books	POST /api/books
Route Type	Static	Static
Intent	Fetch data	Send/Create data
Server Action	Maps to a "Read" handler	Maps to a "Create" handler
Variable Parts?	None	None

Dynamic Routes

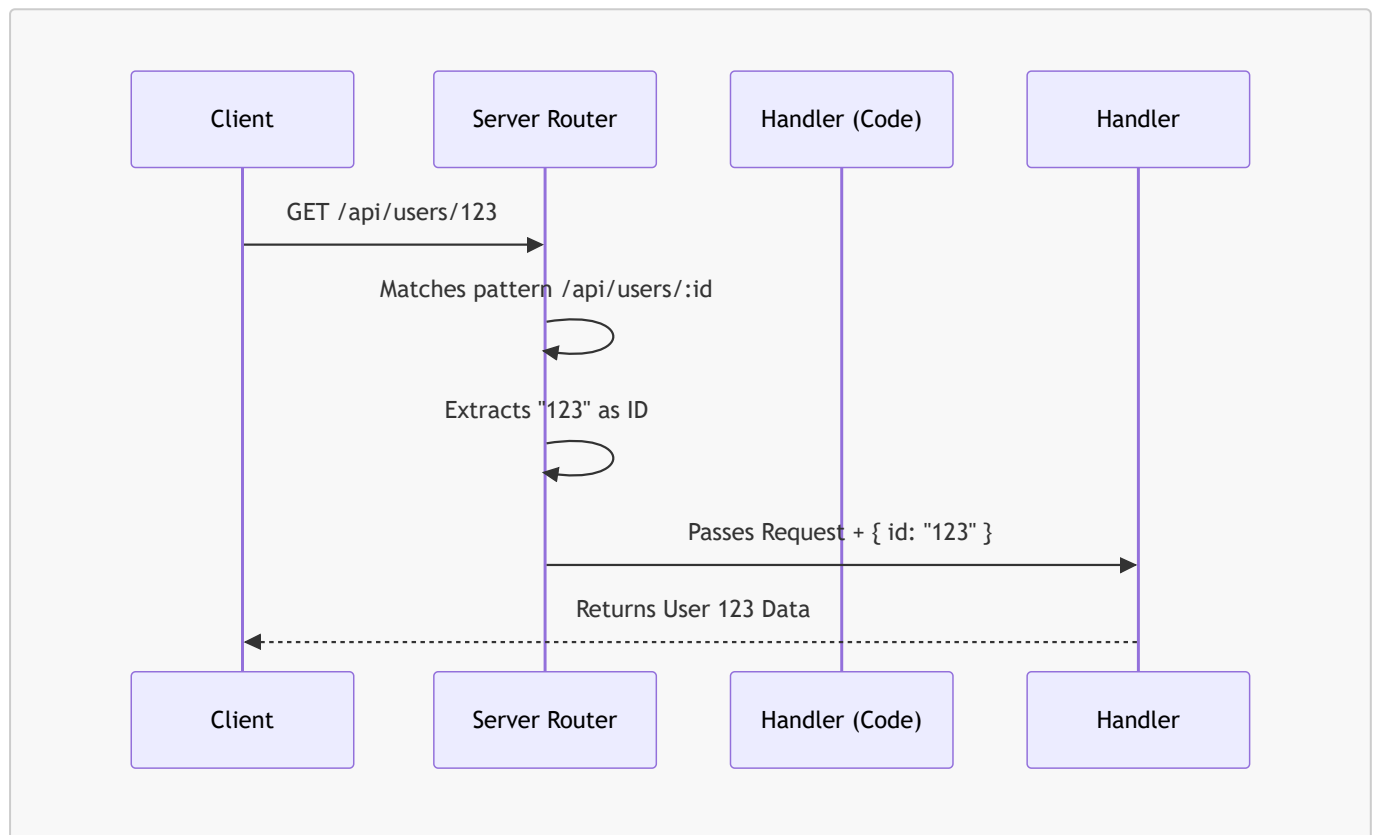
Static routes are great for broad resources, but what if you want to look at just *one* specific book or user? You can't hardcode a static route for every single user in your database (e.g., `/api/users/1`, `/api/users/2`, etc.). That would be impossible!

This is where **Dynamic Routes** come in.

Dynamic routes use a placeholder or variable inside the URL path. In modern backend frameworks (like Node.js, Go, or Python), this is typically represented with a colon `:`.

For example, you might define a route as `/api/users/:id`.

- The `/api/users/` part is static.
- The `:id` part is dynamic. It tells the server, "Expect *any* string or number here, and whatever it is, extract it and call it `id`."



When a request comes in for `/api/users/123`, the server's routing engine catches it, pulls out the `123`, and hands it over to your code so you can query the database for that specific user. Everything in the URL is treated as a string, so your code will receive `"123"` and can process it from there.

API Versioning: Upgrading Without Breaking Things

As your application grows, your API needs will change. Let's say you built your backend originally for a web application, and a specific route returns a user object like this: `{ id, name, price }`. But later, your company decides to launch a mobile app, and the mobile team requires the data slightly differently, perhaps as `{ id, title, price }`.

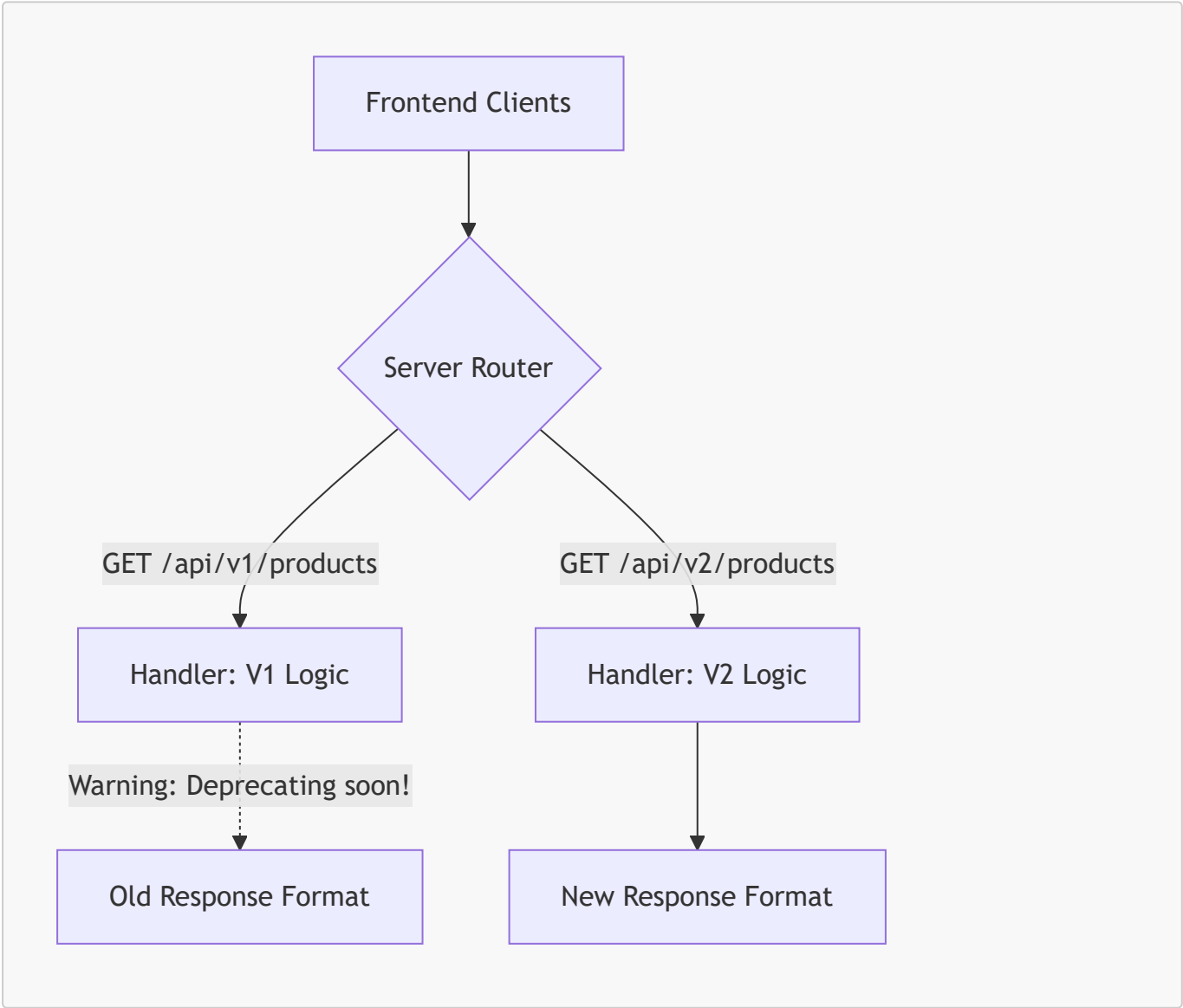
How do you change the response without completely breaking the existing web application that relies on the old format? You use **Route Versioning**.

Instead of creating entirely new, messy route paths (like `/api/new-products`), you add a version number right into the URL path.

- Version 1: `/api/v1/products`
- Version 2: `/api/v2/products`

By doing this, you communicate your intent clearly. The web app can continue safely calling `v1`, while the new mobile app can start using `v2`. This gives your frontend engineers a comfortable window to migrate their code over to `v2`. Once everyone has migrated, you can safely deprecate and remove `v1`.

Feature	v1 Endpoint	v2 Endpoint
Example Path	<code>/api/v1/products</code>	<code>/api/v2/products</code>
Data Returned	<code>id, name, price</code>	<code>id, title, price</code>
Status	Deprecated (Phase-out)	Active (New Standard)
Primary Benefit	Keeps old clients working	Supports new business requirements



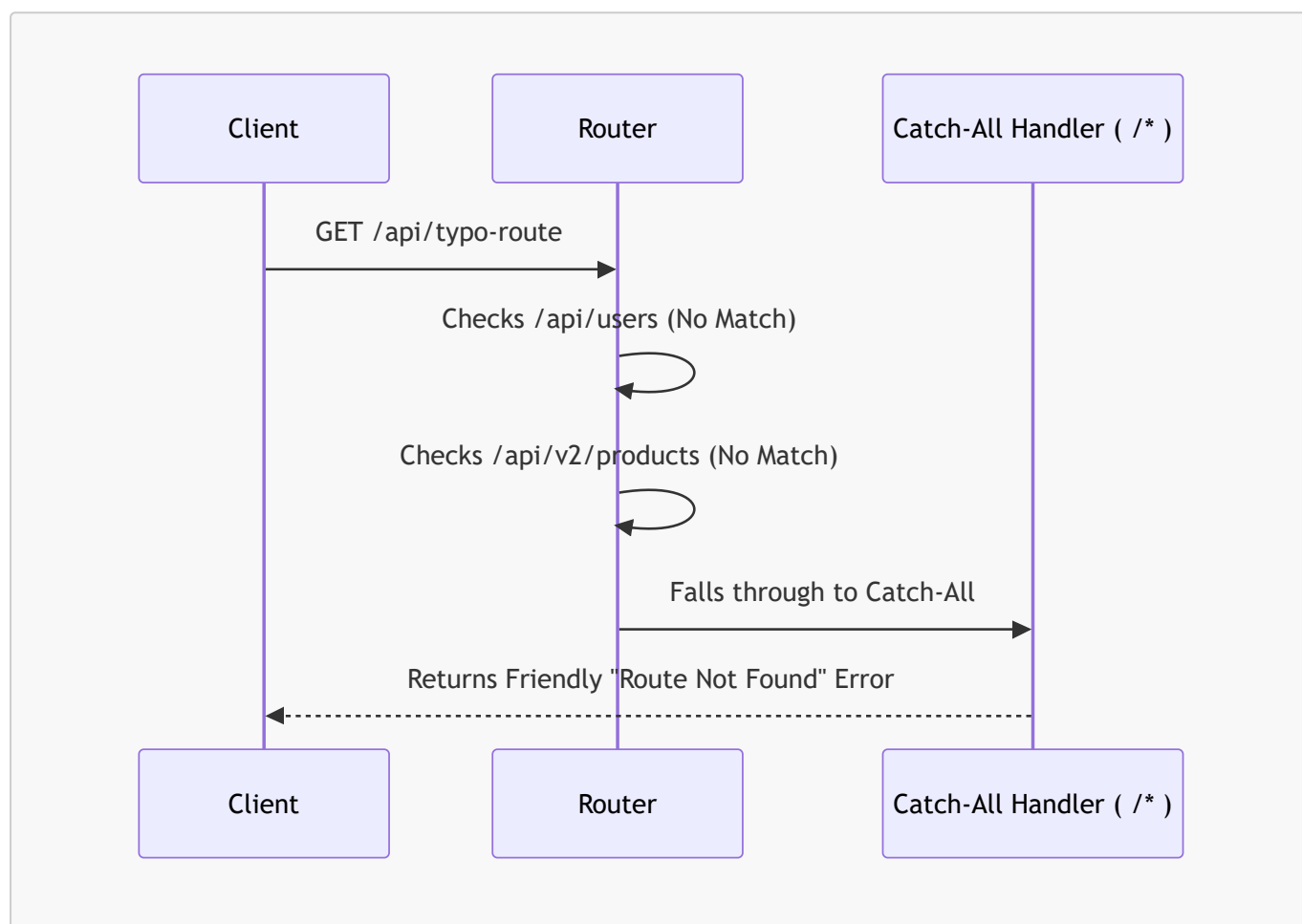
Catch-All Routes: Your API's Safety Net

What happens if a user or a developer makes a typo and requests a route that simply doesn't exist? For example, they ask for `/api/v3/products` when you only have up to `v2`.

By default, a server might just return a blank, unhelpful "null" response or simply hang up the connection. To provide a better developer and user experience, we use a **Catch-All Route**.

In your routing code, after you have defined every single valid route, you place one final, overarching route at the very bottom. It usually looks something like `/*` (slash star).

Because the server checks routes sequentially from top to bottom, if a request bypasses all your valid routes without finding a match, it falls into this catch-all bucket. The catch-all handler then sends back a friendly, structured message saying something like, "Sorry, this route was not found!"



Actionable Takeaways

- **Separate the 'What' from the 'Where':** Remember that the HTTP Method is your action (GET, POST), while the route URL is your target resource. Use them together to create clean, predictable APIs.
- **Keep Static Routes Simple:** Use static routes for broad resource actions (e.g., getting a list of items or creating a new item).
- **Use Colons for Dynamics:** When you need to fetch specific items, use dynamic route parameters (like `:id`). Remember that your backend will receive these parameters as strings, so you may need to parse them into numbers if your database requires it.
- **Version Early:** Don't be afraid to put `/v1/` in your routes right from the start. It saves you from massive headaches when you eventually need to change your data structures without breaking live apps.

- **Always Catch Your Fallbacks:** Never let your API fail silently. Always implement a `/*` catch-all route at the bottom of your router to handle typos and missing endpoints gracefully.